

A Distributional Capacity Framework for a Long-Only Equity Strategy

ADV Migration, Execution Horizons, Market Impact and Alpha Decay

Methodology and Toy Implementation

25 September 2026

Generated at 21:34:17 BST

Abstract

This note extends a first-generation capacity model in two directions. First, it replaces deterministic migration of representative trades across ADV buckets with a continuous, heavy-tailed distribution of parent-order participation. Second, it links the execution horizon to participation and allows expected alpha to decay while a parent order is worked. The model therefore captures the central capacity trade-off: slower trading reduces effective daily participation and market impact, but delays implementation and sacrifices time-sensitive alpha. The implementation remains deliberately transparent and is intended as a bridge to a trade-level production model.

Contents

1	Objective and conventions	2
2	Strategy economics	2
3	Why a Burr XII participation distribution?	2
4	AUM-dependent distributional migration	2
5	Distribution calibration	4
6	Impact curve	4
7	Execution horizon	5
8	Alpha decay during execution	5
9	Net capacity equations	6
10	Illustrative scenarios	8
11	Calibration priorities	8
12	Limitations	9
13	Python implementation	9
14	Summary	15

1 Objective and conventions

The model maps AUM into a distribution of parent-order sizes as a fraction of average daily volume (ADV), then maps each order into an execution horizon, market impact and alpha-capture factor:

$$A \longrightarrow F_A(p) \longrightarrow D(p) \longrightarrow \{\text{impact, alpha capture}\} \longrightarrow R_{\text{net}}(A).$$

Turnover T is one-way annual turnover. Annual executed notional, counting purchases and sales, is approximated by $2TA$. Impact cost is expressed in basis points per executed notional.

The supplied bucket shares are shares of executed notional. They are not trade-count frequencies. Consequently, the fitted participation distribution is a notional-weighted distribution. A trade-frequency model requires order counts or an explicit model of trade size conditional on participation.

2 Strategy economics

Let R_g denote current gross annualised relative return, TE tracking error, N average holdings and T one-way turnover. Baseline diagnostics remain

$$IR_g = \frac{R_g}{TE}, \quad \alpha_{\text{holding}} = \frac{R_g}{N}, \quad \alpha_{\text{cycle}} = \frac{R_g}{T}, \quad H \approx \frac{1}{T}.$$

The extension changes R_g away from current AUM because implementation delay changes the fraction of expected alpha captured.

3 Why a Burr XII participation distribution?

A pure Pareto distribution is useful for an upper tail, but it imposes a hard lower cutoff and a single tail shape. That is restrictive when the observed data include both many small trades and a thin but economically important upper tail. Empirical studies find power-law tails in trade-size distributions [1]. We therefore use a Burr XII distribution, whose CDF is

$$F(p; c, d, \lambda) = 1 - \left[1 + \left(\frac{p}{\lambda} \right)^c \right]^{-d}, \quad p > 0.$$

It has a flexible body and a Pareto-like survival tail

$$1 - F(p) \sim p^{-cd}.$$

The model fits c , d and λ to cumulative notional shares at the observed ADV bucket edges. It regularises the tail index to $cd \geq 1.25$. This preserves a heavy tail while ensuring that mean participation exists. The chosen bound is an explicit modelling assumption and should be replaced by trade-level estimation when possible.

4 AUM-dependent distributional migration

Instead of multiplying every bucket representative by one scale factor and moving its entire mass at once, the model shifts a continuous distribution. Its scale parameter evolves as

$$\lambda(A) = \lambda_0 \left(\frac{A}{A_0} \right)^\eta,$$

where η is the participation-scale elasticity. The toy calibration uses $\eta = 0.85$: participation rises with AUM, but portfolio and execution adaptation damp the mechanical increase. Setting $\eta = 1$ restores proportional scaling.

An optional tail-composition parameter κ changes the second Burr shape parameter:

$$d(A) = d_0 \left(\frac{A}{A_0} \right)^{-\kappa}.$$

Positive κ makes the upper tail heavier as AUM increases. It should not be used merely to represent larger orders; that role belongs to $\lambda(A)$. It represents a structural change in the portfolio's liquidity composition. The baseline sets $\kappa = 0$.

For a bucket with lower and upper bounds $(\ell_i, u_i]$, the migrated share is

$$w_i(A) = F_A(u_i) - F_A(\ell_i).$$

Migration is therefore smooth rather than occurring in discrete jumps.

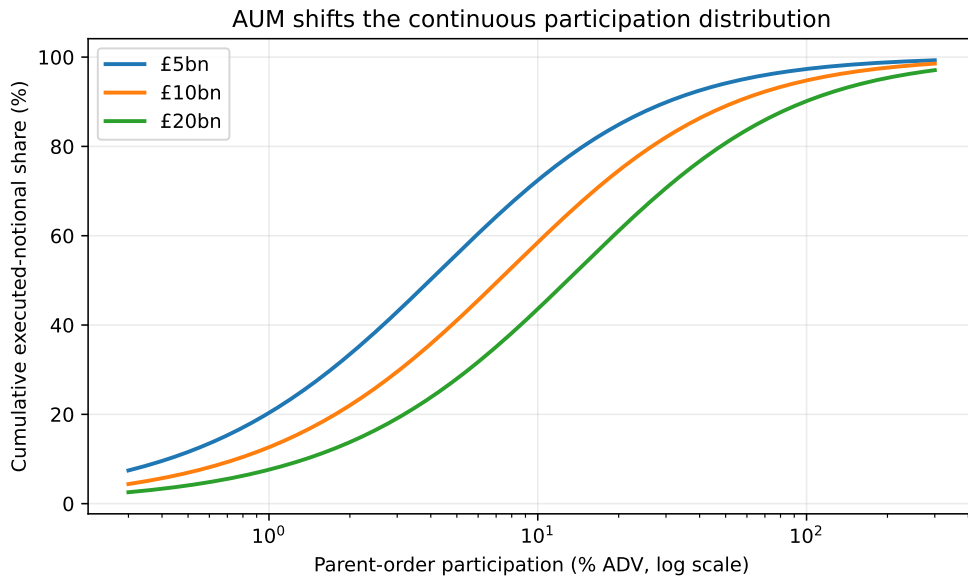


Figure 1: AUM shifts the fitted continuous participation distribution.

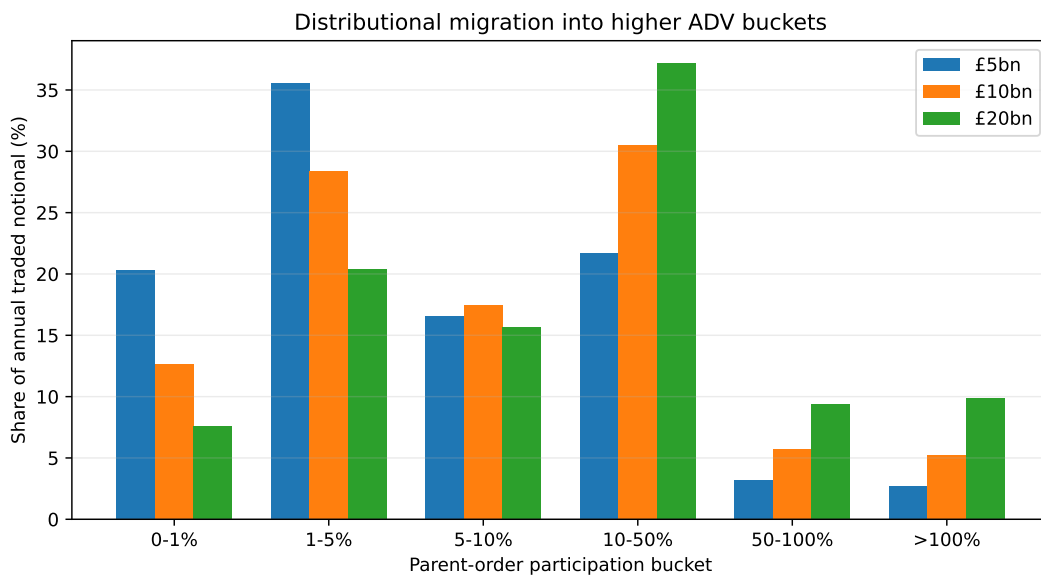


Figure 2: Smooth distributional migration across ADV participation buckets.

5 Distribution calibration

Table 1 compares the supplied notional shares with the regularised Burr XII fit. The fit intentionally does not reproduce every bucket exactly: a continuous three-parameter distribution and a finite-mean tail constraint impose cross-bucket discipline.

Table 1: Observed and fitted executed-notional shares at current AUM

ADV bucket	Observed	Burr XII fit
0–1%	20.00%	20.33%
1–5%	35.00%	35.55%
5–10%	20.00%	16.54%
10–50%	15.00%	21.72%
50–100%	7.00%	3.20%
>100%	3.00%	2.66%

The fitted parameters are approximately

$$c = 0.9437, \quad d = 1.3245, \quad \lambda_0 = 0.0590, \quad cd = 1.25.$$

With only six aggregate buckets, parameter uncertainty is substantial. Production reporting should include confidence intervals and compare Burr XII, lognormal-body/Pareto-tail and empirical-CDF specifications out of sample.

6 Impact curve

Expected bucket costs are smoothed using

$$c(p) = a + bp^\gamma,$$

with positive parameters estimated by nonlinear least squares. Figure 3 shows the toy fit. Evidence on large metaorders often supports concave impact and a square-root benchmark [2]; the fitted exponent here is data-driven and remains illustrative.

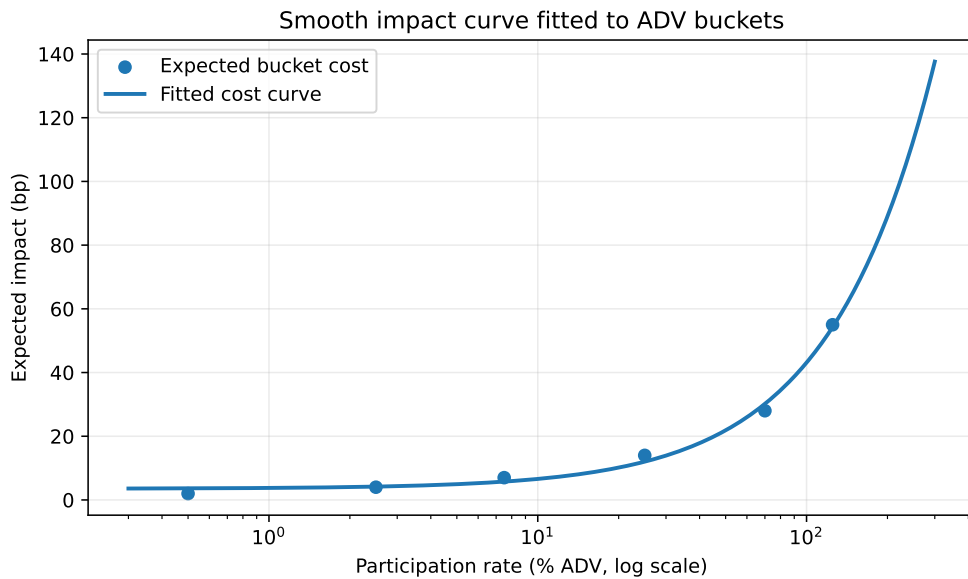


Figure 3: Smooth impact curve fitted to the original ADV-bucket observations.

7 Execution horizon

Let ρ be the target daily participation rate and $D_{\max}$ a maximum horizon. The toy rule is

$$D(p) = \min\left(D_{\max}, \max\left(1, \left\lceil \frac{p}{\rho} \right\rceil\right)\right).$$

The baseline uses $\rho = 10\%$ ADV and $D_{\max} = 10$ trading days. This is a policy rule, not an estimated optimum. Optimal-execution research formalises the trade-off between trading costs and the risk of slow execution [3].

Because current observed costs already embed the current execution schedule, the counterfactual effective participation is measured relative to that schedule:

$$p_{\text{eff},j}(A) = p_j(A) \frac{D_j(A_0)}{D_j(A)}.$$

At current AUM, $p_{\text{eff},j}(A_0) = p_j(A_0)$, so the model reproduces the observed baseline weighted impact. When the horizon lengthens, effective daily intensity increases less rapidly than parent-order participation.

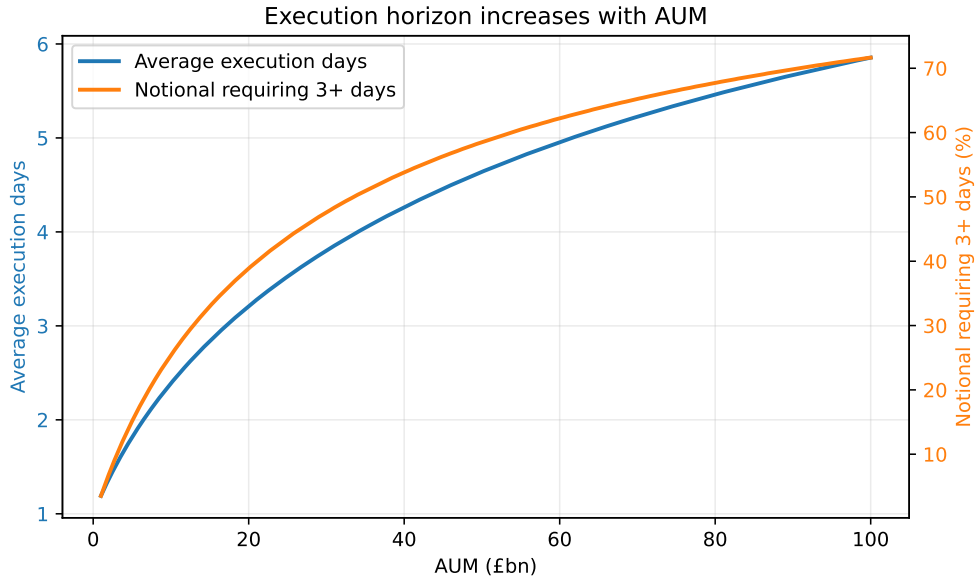


Figure 4: Average horizon and the share of notional requiring at least three days.

8 Alpha decay during execution

Let expected alpha have half-life H_α trading days, with exponential decay rate $\delta = \log(2)/H_\alpha$. If an order is executed in equal daily slices over D days, its average alpha-capture factor is

$$g(D, H_\alpha) = \frac{1}{D} \sum_{t=0}^{D-1} e^{-\delta t} = \frac{1 - e^{-\delta D}}{D(1 - e^{-\delta})}.$$

The baseline uses $H_\alpha = 10$ trading days. Because the observed 4% gross alpha already reflects the current implementation schedule, gross alpha is normalised at current AUM:

$$R_g(A) = R_g(A_0) \mathbb{E} \left[\frac{g(D_j(A), H_\alpha)}{g(D_j(A_0), H_\alpha)} \right].$$

The expectation is currently notional-weighted, which assumes alpha contribution is proportional to executed notional. A production model should use trade-level forecast alpha weights. Research on predictable returns with costs shows that signals with different mean-reversion speeds imply different desired trading speeds [4]; institutional evidence also documents gradual post-trade alpha decay [5].

9 Net capacity equations

For the distribution at target AUM,

$$\bar{c}(A) = \mathbb{E}[c(p_{\text{eff}}(A))], \quad C(A) = \frac{2T\bar{c}(A)}{10,000}.$$

Net alpha and information ratio are

$$R_{\text{net}}(A) = R_g(A) - C(A), \quad IR_{\text{net}}(A) = \frac{R_{\text{net}}(A)}{TE}.$$

This introduces an explicit economic trade-off: extending the horizon reduces the rate at which impact rises, but it lowers implemented gross alpha.

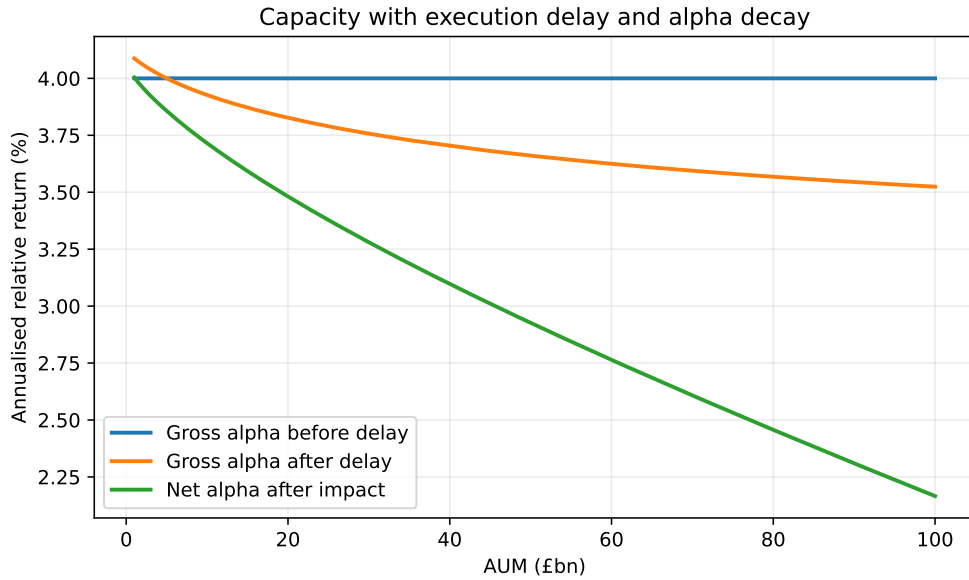


Figure 5: Gross alpha before delay, gross alpha after delay, and net alpha after impact.

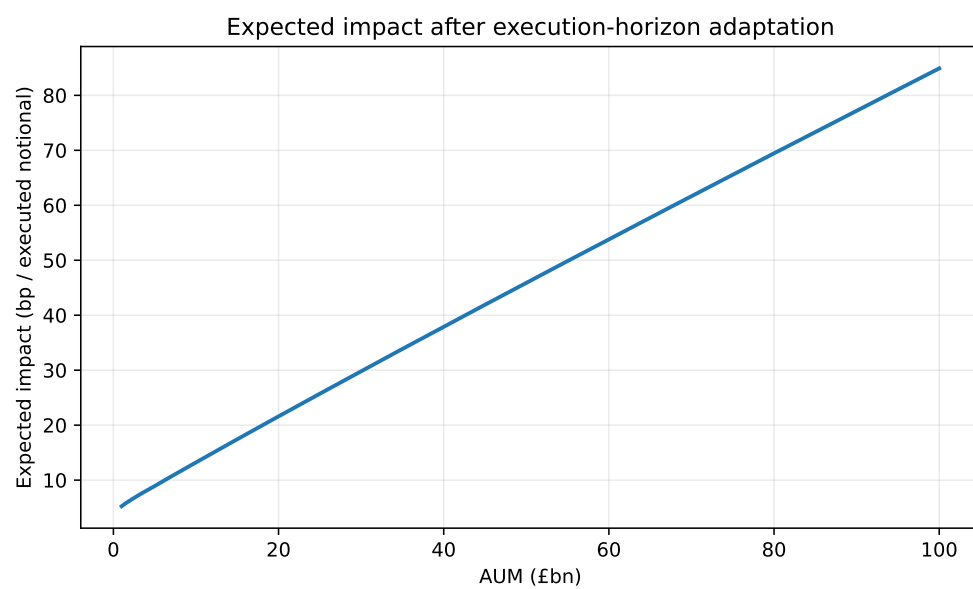


Figure 6: Expected impact after execution-horizon adaptation.

10 Illustrative scenarios

Table 2 reports the new model at three AUM levels.

Table 2: Distributional capacity scenarios

Metric	£5bn	£10bn	£20bn
Gross alpha before delay	4.000%	4.000%	4.000%
Alpha captured	100.00%	98.17%	95.67%
Gross alpha after delay	4.000%	3.927%	3.827%
Weighted expected impact	8.91 bp	13.22 bp	21.60 bp
Annual impact drag	14.26 bp	21.16 bp	34.56 bp
Net alpha	3.857%	3.715%	3.481%
Net IR	0.964	0.929	0.870
Average execution days	1.81	2.39	3.21
Notional requiring multiple days	27.58%	41.48%	56.40%
Notional requiring at least 3 days	15.13%	25.40%	38.90%

Under the stated toy assumptions, continuous root solving gives:

- alpha capture falls to 95% at approximately £23.51bn;
- alpha retained after delay and impact falls to 80% at approximately £34.29bn;
- net IR reaches 0.75 at approximately £45.62bn;
- annual impact drag reaches 100 bp at approximately £71.08bn.

These are scenario outputs, not confidence bounds. They depend materially on η , κ , ρ , $D_{\max}$ and H_α .

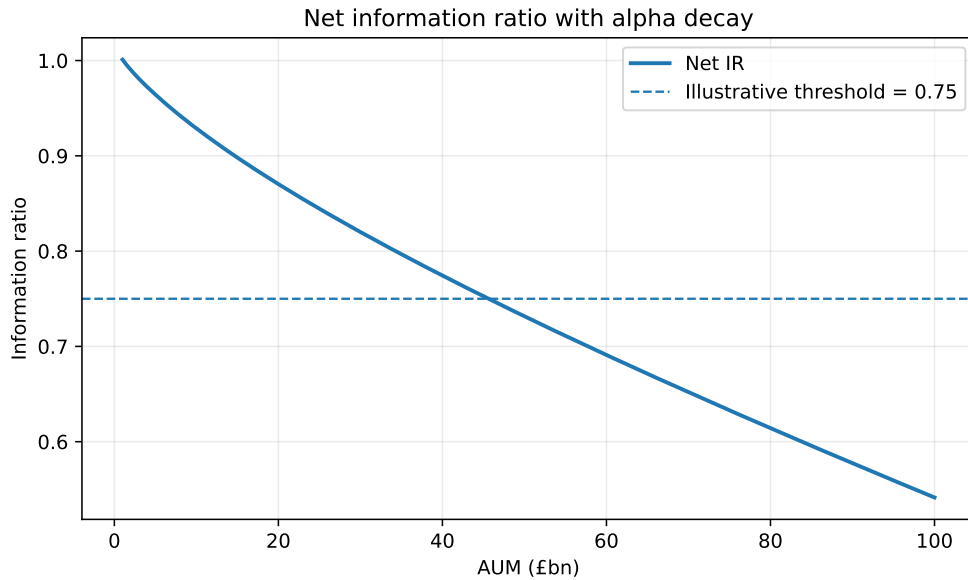


Figure 7: Net information ratio after distributional migration, delay and impact.

11 Calibration priorities

The most important production inputs are:

1. parent-order participation and executed-notional weights;
2. actual execution duration and daily participation schedules;
3. expected and realised impact conditional on participation and duration;
4. trade-level forecast alpha and its decay by signal family;
5. AUM elasticity η estimated from portfolio rescaling exercises;
6. tail-composition drift κ estimated only if portfolio composition changes;
7. uncertainty intervals and alternative distributional specifications.

The execution horizon should eventually be chosen by optimisation rather than a fixed daily cap:

$$D_j^* = \arg \max_D \left\{ \text{captured alpha}_j(D) - \text{impact}_j(D) - \text{execution risk}_j(D) \right\}.$$

12 Limitations

The aggregate bucket data do not identify a unique participation distribution. The Burr XII fit is a disciplined interpolation and tail model, not a discovered law. The current model also assumes notional share equals alpha share, uses one alpha half-life for all signals, holds tracking error constant, and treats the impact curve as transportable across execution horizons after a simple baseline-relative adjustment. Each assumption is exposed as a parameter or a clear replacement point for trade-level data.

13 Python implementation

The accompanying `long_only_capacity_tool.py` uses the project's `pyx86` Conda environment. It fits the Burr XII and impact curves, produces smooth bucket migration, calculates execution horizons and alpha capture, solves capacity thresholds continuously, writes CSV outputs and generates all figures in this report.

```

"""Distributional long-only equity capacity model.

The model combines four mechanisms:

1. a smooth market-impact curve fitted to observed ADV buckets;
2. a continuous Burr XII distribution for parent-order participation;
3. an AUM-dependent scale and optional tail-shape change in that distribution;
4. execution-horizon choice and exponential alpha decay.

Conventions
-----
- turnover is ONE-WAY annual turnover, T;
- executed notional (buys + sells) is approximately 2*T*AUM;
- impact is in basis points per executed notional;
- current gross alpha is normalized after the current execution schedule;
- bucket shares are treated as shares of executed notional, not trade counts.

This remains a transparent toy model. Production calibration should use
trade-level parent orders, realised execution horizons and signal-decay data.
"""

from __future__ import annotations

from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

```

```

from scipy.optimize import brentq, curve_fit, least_squares
from scipy.stats import burr12

OUT = Path(__file__).resolve().parent

# -----
# 1) Strategy and implementation assumptions
# -----
AUMO_BN = 5.0
GROSS_ALPHA = 0.040
TRACKING_ERROR = 0.040
TURNOVER_1W = 0.80
N_HOLDINGS = 100

# Distributional migration assumptions.
#  $scale(A) = scale(A0) * (A/A0)^{PARTICIPATION\_SCALE\_ELASTICITY}$ 
# An elasticity below one allows execution/portfolio adaptation to damp the
# mechanical increase in participation. Set it to one for proportional scaling.
PARTICIPATION_SCALE_ELASTICITY = 0.85

# Positive values make the upper tail heavier as AUM increases by reducing the
# second Burr shape parameter. Keep at zero unless data support composition drift.
TAIL_THICKENING_ELASTICITY = 0.00

# Execution and alpha-decay assumptions.
TARGET_DAILY_PARTICIPATION = 0.10 # 10% ADV per execution day
MAX_EXECUTION_DAYS = 10
ALPHA_HALF_LIFE_DAYS = 10.0
N_DISTRIBUTION_QUANTILES = 4000

buckets = pd.DataFrame({
    "bucket": ["0-1%", "1-5%", "5-10%", "10-50%", "50-100%", ">100%"],
    "lower": [0.00, 0.01, 0.05, 0.10, 0.50, 1.00],
    "upper": [0.01, 0.05, 0.10, 0.50, 1.00, np.inf],
    "p_adv": [0.005, 0.025, 0.075, 0.25, 0.70, 1.25],
    "notional_share": [0.20, 0.35, 0.20, 0.15, 0.07, 0.03],
    "expected_bp": [2.0, 4.0, 7.0, 14.0, 28.0, 55.0],
    "realised_bp": [2.2, 4.4, 7.8, 16.0, 34.0, 72.0],
})
assert np.isclose(buckets["notional_share"].sum(), 1.0)

# -----
# 2) Smooth impact curves
# -----
def impact_curve(p, a, b, gamma):
    """Impact in bp as a function of effective participation p."""
    return a + b * np.power(p, gamma)

def fit_impact_curve(df: pd.DataFrame, col: str = "expected_bp"):
    x = df["p_adv"].to_numpy(float)
    y = df[col].to_numpy(float)
    popt, _ = curve_fit(
        impact_curve,
        x,
        y,
        p0=(0.5, 45.0, 0.6),
        bounds=([0.0, 0.0, 0.10], [20.0, 500.0, 1.50]),
        maxfev=100000,
    )
    return popt

PARAM_EXPECTED = fit_impact_curve(buckets, "expected_bp")
PARAM_REALISED = fit_impact_curve(buckets, "realised_bp")

# -----
# 3) Continuous participation distribution and bucket migration
# -----

```

```

def fit_participation_distribution(df: pd.DataFrame):
    """Fit Burr XII parameters to cumulative notional shares at bucket edges.

    Burr XII has a flexible body and a Pareto-like upper tail:
         $F(p) = 1 - [1 + (p / \text{scale})^c]^{-d}$ .
    """
    finite = np.isfinite(df["upper"].to_numpy(float))
    edges = df.loc[finite, "upper"].to_numpy(float)
    targets = df.loc[finite, "notional_share"].cumsum().to_numpy(float)

    def residuals(log_params):
        # Parameterise the tail index directly. For Burr XII the survival tail
        # is proportional to  $p^{(-c*d)}$ . Requiring  $c*d \geq 1.25$  gives a finite
        # mean while retaining the empirically common 1-to-2 heavy-tail range.
        c, tail_index, scale = np.exp(log_params)
        d = tail_index / c
        return burr12.cdf(edges, c, d, scale=scale) - targets

    result = least_squares(
        residuals,
        np.log([1.0, 1.50, 0.03]),
        bounds=(np.log([0.10, 1.25, 1e-4]), np.log([10.0, 20.0, 10.0])),
        max_nfev=100000,
    )
    if not result.success:
        raise RuntimeError(f"Burr XII calibration failed: {result.message}")
    c, tail_index, scale = np.exp(result.x)
    return c, tail_index / c, scale

BURR_CO, BURR_DO, BURR_SCALE0 = fit_participation_distribution(buckets)
QUANTILE_GRID = (np.arange(N_DISTRIBUTION_QUANTILES) + 0.5) / N_DISTRIBUTION_QUANTILES

def participation_parameters(aum_bn: float):
    """Return Burr XII parameters at a target AUM."""
    if aum_bn <= 0:
        raise ValueError("AUM must be positive")
    aum_ratio = aum_bn / AUMO_BN
    scale = BURR_SCALE0 * aum_ratio ** PARTICIPATION_SCALE_ELASTICITY
    d = BURR_DO * aum_ratio ** (-TAIL_THICKENING_ELASTICITY)
    return BURR_CO, d, scale

def participation_cdf(p, aum_bn: float):
    c, d, scale = participation_parameters(aum_bn)
    return burr12.cdf(p, c, d, scale=scale)

def participation_samples(aum_bn: float):
    """Deterministic notional-weighted participation quantiles."""
    c, d, scale = participation_parameters(aum_bn)
    return burr12.ppf(QUANTILE_GRID, c, d, scale=scale)

def migrated_bucket_shares(aum_bn: float) -> pd.Series:
    """Smooth bucket shares implied by the fitted distribution at target AUM."""
    shares = []
    for row in buckets.itertuples(index=False):
        lower_cdf = 0.0 if row.lower == 0 else participation_cdf(row.lower, aum_bn)
        upper_cdf = 1.0 if not np.isfinite(row.upper) else participation_cdf(row.upper, aum_bn)
        shares.append(float(upper_cdf - lower_cdf))
    return pd.Series(shares, index=buckets["bucket"], dtype=float)

# -----
# 4) Execution horizon, alpha decay and impact
# -----

def execution_days(participation):
    """Execution days implied by a target daily participation rate."""
    days = np.ceil(np.asarray(participation) / TARGET_DAILY_PARTICIPATION)
    return np.clip(days, 1, MAX_EXECUTION_DAYS).astype(int)

```

```

def alpha_capture_factor(days, half_life_days: float = ALPHA_HALF_LIFE_DAYS):
    """Average exponential alpha remaining across equally sized daily slices."""
    if half_life_days <= 0:
        raise ValueError("Alpha half-life must be positive")
    days = np.asarray(days, dtype=float)
    decay = np.log(2.0) / half_life_days
    numerator = 1.0 - np.exp(-decay * days)
    denominator = days * (1.0 - np.exp(-decay))
    return numerator / denominator

BASE_PARTICIPATION = participation_samples(AUMO_BN)
BASE_EXECUTION_DAYS = execution_days(BASE_PARTICIPATION)
BASE_ALPHA_CAPTURE = alpha_capture_factor(BASE_EXECUTION_DAYS)

def distribution_state(aum_bn: float):
    participation = participation_samples(aum_bn)
    days = execution_days(participation)

    # Current observed impact already embeds the current implementation schedule.
    # This ratio changes effective daily intensity relative to that baseline.
    effective_participation = participation * BASE_EXECUTION_DAYS / days

    capture = alpha_capture_factor(days)
    relative_alpha_capture = capture / BASE_ALPHA_CAPTURE
    return participation, days, effective_participation, relative_alpha_capture

RAW_EXPECTED_IMPACT_BP = float(np.dot(buckets["notional_share"], buckets["expected_bp"]))
RAW_REALISED_IMPACT_BP = float(np.dot(buckets["notional_share"], buckets["realised_bp"]))
BASE_EXPECTED_MODEL_BP = float(np.mean(impact_curve(BASE_PARTICIPATION, *PARAM_EXPECTED)))
BASE_REALISED_MODEL_BP = float(np.mean(impact_curve(BASE_PARTICIPATION, *PARAM_REALISED)))
EXPECTED_IMPACT_CALIBRATION = RAW_EXPECTED_IMPACT_BP / BASE_EXPECTED_MODEL_BP
REALISED_IMPACT_CALIBRATION = RAW_REALISED_IMPACT_BP / BASE_REALISED_MODEL_BP

def weighted_impact_bp(aum_bn: float, realised: bool = False) -> float:
    _, _, effective_participation, _ = distribution_state(aum_bn)
    if realised:
        raw = np.mean(impact_curve(effective_participation, *PARAM_REALISED))
        return float(raw * REALISED_IMPACT_CALIBRATION)
    raw = np.mean(impact_curve(effective_participation, *PARAM_EXPECTED))
    return float(raw * EXPECTED_IMPACT_CALIBRATION)

def strategy_metrics(aum_bn: float) -> dict:
    participation, days, effective_participation, relative_capture = distribution_state(aum_bn)
    c_bp = float(
        np.mean(impact_curve(effective_participation, *PARAM_EXPECTED))
        * EXPECTED_IMPACT_CALIBRATION
    )
    drag = 2.0 * TURNOVER_1W * c_bp / 10000.0

    # In the toy model, alpha share is proportional to executed-notional share.
    alpha_capture = float(np.mean(relative_capture))
    implemented_gross_alpha = GROSS_ALPHA * alpha_capture
    net_alpha = implemented_gross_alpha - drag

    return {
        "aum_bn": aum_bn,
        "baseline_gross_alpha_pct": GROSS_ALPHA * 100.0,
        "alpha_capture_pct": alpha_capture * 100.0,
        "implemented_gross_alpha_pct": implemented_gross_alpha * 100.0,
        "weighted_impact_bp": c_bp,
        "annual_drag_bp": drag * 10000.0,
        "net_alpha_pct": net_alpha * 100.0,
        "gross_ir_after_delay": implemented_gross_alpha / TRACKING_ERROR,
        "net_ir": net_alpha / TRACKING_ERROR,
        "alpha_retained_pct": net_alpha / GROSS_ALPHA * 100.0,
        "average_execution_days": float(np.mean(days)),
        "share_multiday_pct": float(np.mean(days >= 2) * 100.0),
    }

```

```

        "share_3plus_days_pct": float(np.mean(days >= 3) * 100.0),
        "mean_parent_participation_pct": float(np.mean(participation) * 100.0),
    }

# -----
# 5) Outputs and diagnostics
# -----
alpha_per_holding_slot = GROSS_ALPHA / N_HOLDINGS
position_replacements = TURNOVER_1W * N_HOLDINGS
portfolio_contribution_per_replacement = GROSS_ALPHA / position_replacements
alpha_per_capital_cycle = GROSS_ALPHA / TURNOVER_1W
average_holding_years = 1.0 / TURNOVER_1W

summary = pd.DataFrame([
    ["Gross relative return", GROSS_ALPHA * 100, "% p.a."],
    ["Tracking error", TRACKING_ERROR * 100, "% p.a."],
    ["One-way turnover", TURNOVER_1W * 100, "% p.a."],
    ["Average holdings", N_HOLDINGS, "count"],
    ["Alpha per holding slot", alpha_per_holding_slot * 10000, "bp p.a."],
    ["Portfolio contribution/replacement", portfolio_contribution_per_replacement * 10000, "bp"],
    ["Alpha per unit capital recycled", alpha_per_capital_cycle * 100, "% per cycle"],
    ["Approx. average holding period", average_holding_years, "years"],
    ["Participation scale elasticity", PARTICIPATION_SCALE_ELASTICITY, "x"],
    ["Tail-thickening elasticity", TAIL_THICKENING_ELASTICITY, "x"],
    ["Target daily participation", TARGET_DAILY_PARTICIPATION * 100, "% ADV"],
    ["Alpha half-life", ALPHA_HALF_LIFE_DAYS, "trading days"],
], columns=["Metric", "Value", "Unit"])
summary.to_csv(OUT / "toy_strategy_summary.csv", index=False)

distribution_fit = buckets[["bucket", "notional_share"]].copy()
distribution_fit["burr_fitted_share"] = migrated_bucket_shares(AUMO_BN).values
distribution_fit.to_csv(OUT / "toy_distribution_fit.csv", index=False)

scenario_aums = [5.0, 10.0, 20.0]
scenarios = pd.DataFrame([strategy_metrics(a) for a in scenario_aums])
scenarios.to_csv(OUT / "toy_aum_scenarios.csv", index=False)

migration = pd.DataFrame({
    "bucket": buckets["bucket"],
    "share_5bn": migrated_bucket_shares(5.0).values,
    "share_10bn": migrated_bucket_shares(10.0).values,
    "share_20bn": migrated_bucket_shares(20.0).values,
})
migration.to_csv(OUT / "toy_bucket_migration.csv", index=False)

aum_grid = np.linspace(1.0, 100.0, 397)
curve = pd.DataFrame([strategy_metrics(a) for a in aum_grid])
curve.to_csv(OUT / "toy_capacity_curve.csv", index=False)

def exact_crossing(metric: str, threshold: float, direction: str = "below"):
    values = curve[metric].to_numpy(float)
    condition = values <= threshold if direction == "below" else values >= threshold
    indices = np.where(condition)[0]
    if len(indices) == 0:
        return None
    idx = int(indices[0])
    if idx == 0:
        return float(curve.iloc[0]["aum_bn"])
    left = float(curve.iloc[idx - 1]["aum_bn"])
    right = float(curve.iloc[idx]["aum_bn"])
    return float(brentq(lambda a: strategy_metrics(a)[metric] - threshold, left, right))

thresholds = {
    "AUM where net IR <= 0.75": exact_crossing("net_ir", 0.75),
    "AUM where alpha retained <= 80%": exact_crossing("alpha_retained_pct", 80.0),
    "AUM where annual impact drag >= 100 bp": exact_crossing("annual_drag_bp", 100.0, "above"),
    "AUM where alpha capture <= 95%": exact_crossing("alpha_capture_pct", 95.0),
}
pd.DataFrame(list(thresholds.items()), columns=["Threshold", "AUM_bn"]).to_csv(
    OUT / "toy_capacity_thresholds.csv", index=False
)

```

```

)

# -----
# 6) Figures
# -----
p_grid = np.geomspace(0.003, 3.0, 250)
plt.figure(figsize=(7.2, 4.4))
plt.scatter(buckets["p_adv"] * 100, buckets["expected_bp"], label="Expected bucket cost")
plt.plot(p_grid * 100, impact_curve(p_grid, *PARAM_EXPECTED), label="Fitted cost curve", linewidth=2)
plt.xscale("log")
plt.xlabel("Participation rate (% ADV, log scale)")
plt.ylabel("Expected impact (bp)")
plt.title("Smooth impact curve fitted to ADV buckets")
plt.legend()
plt.grid(alpha=0.25)
plt.tight_layout()
plt.savefig(OUT / "fig_impact_curve_fit.pdf", bbox_inches="tight")
plt.savefig(OUT / "fig_impact_curve_fit.png", dpi=180, bbox_inches="tight")
plt.close()

plt.figure(figsize=(7.2, 4.4))
for aum in scenario_aums:
    plt.plot(
        p_grid * 100,
        participation_cdf(p_grid, aum) * 100,
        linewidth=2,
        label=f"£{aum:g}bn",
    )
plt.xscale("log")
plt.xlabel("Parent-order participation (% ADV, log scale)")
plt.ylabel("Cumulative executed-notional share (%)")
plt.title("AUM shifts the continuous participation distribution")
plt.legend()
plt.grid(alpha=0.25)
plt.tight_layout()
plt.savefig(OUT / "fig_participation_distribution.pdf", bbox_inches="tight")
plt.savefig(OUT / "fig_participation_distribution.png", dpi=180, bbox_inches="tight")
plt.close()

x = np.arange(len(migration))
w = 0.25
plt.figure(figsize=(8.2, 4.6))
plt.bar(x - w, migration["share_5bn"] * 100, width=w, label="£5bn")
plt.bar(x, migration["share_10bn"] * 100, width=w, label="£10bn")
plt.bar(x + w, migration["share_20bn"] * 100, width=w, label="£20bn")
plt.xticks(x, migration["bucket"])
plt.ylabel("Share of annual traded notional (%)")
plt.xlabel("Parent-order participation bucket")
plt.title("Distributional migration into higher ADV buckets")
plt.legend()
plt.grid(axis="y", alpha=0.25)
plt.tight_layout()
plt.savefig(OUT / "fig_bucket_migration.pdf", bbox_inches="tight")
plt.savefig(OUT / "fig_bucket_migration.png", dpi=180, bbox_inches="tight")
plt.close()

plt.figure(figsize=(7.2, 4.4))
plt.plot(curve["aum_bn"], curve["weighted_impact_bp"], linewidth=2)
plt.xlabel("AUM (£bn)")
plt.ylabel("Expected impact (bp / executed notional)")
plt.title("Expected impact after execution-horizon adaptation")
plt.grid(alpha=0.25)
plt.tight_layout()
plt.savefig(OUT / "fig_impact_vs_aum.pdf", bbox_inches="tight")
plt.savefig(OUT / "fig_impact_vs_aum.png", dpi=180, bbox_inches="tight")
plt.close()

plt.figure(figsize=(7.2, 4.4))
plt.plot(curve["aum_bn"], curve["baseline_gross_alpha_pct"], label="Gross alpha before delay", linewidth=2)
plt.plot(curve["aum_bn"], curve["implemented_gross_alpha_pct"], label="Gross alpha after delay", linewidth=2)
plt.plot(curve["aum_bn"], curve["net_alpha_pct"], label="Net alpha after impact", linewidth=2)

```

```

plt.xlabel("AUM (£bn)")
plt.ylabel("Annualised relative return (%)")
plt.title("Capacity with execution delay and alpha decay")
plt.legend()
plt.grid(alpha=0.25)
plt.tight_layout()
plt.savefig(OUT / "fig_alpha_vs_aum.pdf", bbox_inches="tight")
plt.savefig(OUT / "fig_alpha_vs_aum.png", dpi=180, bbox_inches="tight")
plt.close()

plt.figure(figsize=(7.2, 4.4))
plt.plot(curve["aum_bn"], curve["net_ir"], label="Net IR", linewidth=2)
plt.axhline(0.75, linestyle="--", linewidth=1.2, label="Illustrative threshold = 0.75")
plt.xlabel("AUM (£bn)")
plt.ylabel("Information ratio")
plt.title("Net information ratio with alpha decay")
plt.legend()
plt.grid(alpha=0.25)
plt.tight_layout()
plt.savefig(OUT / "fig_ir_vs_aum.pdf", bbox_inches="tight")
plt.savefig(OUT / "fig_ir_vs_aum.png", dpi=180, bbox_inches="tight")
plt.close()

fig, ax_days = plt.subplots(figsize=(7.2, 4.4))
line_days = ax_days.plot(
    curve["aum_bn"], curve["average_execution_days"],
    label="Average execution days", linewidth=2, color="tab:blue",
)
ax_days.set_xlabel("AUM (£bn)")
ax_days.set_ylabel("Average execution days", color="tab:blue")
ax_days.tick_params(axis="y", labelcolor="tab:blue")
ax_days.grid(alpha=0.25)

ax_share = ax_days.twinx()
line_share = ax_share.plot(
    curve["aum_bn"], curve["share_3plus_days_pct"],
    label="Notional requiring 3+ days", linewidth=2, color="tab:orange",
)
ax_share.set_ylabel("Notional requiring 3+ days (%)", color="tab:orange")
ax_share.tick_params(axis="y", labelcolor="tab:orange")

lines = line_days + line_share
ax_days.legend(lines, [line.get_label() for line in lines], loc="upper left")
ax_days.set_title("Execution horizon increases with AUM")
fig.tight_layout()
fig.savefig(OUT / "fig_execution_horizon.pdf", bbox_inches="tight")
fig.savefig(OUT / "fig_execution_horizon.png", dpi=180, bbox_inches="tight")
plt.close(fig)

print("Expected impact parameters a,b,gamma:", PARAM_EXPECTED)
print("Realised impact parameters a,b,gamma:", PARAM_REALISED)
print("Burr XII parameters c,d,scale:", (BURR_CO, BURR_DO, BURR_SCALE0))
print("Burr tail index c*d:", BURR_CO * BURR_DO)
print("\nDistribution fit:\n", distribution_fit.to_string(index=False))
print("\nScenario table:\n", scenarios.to_string(index=False))
print("\nThresholds:", thresholds)

```

14 Summary

The extended framework replaces discrete representative-bucket jumps with continuous heavy-tailed migration and recognises that liquidity can be obtained by spending time. AUM shifts the participation distribution; the execution policy converts parent-order size into a horizon; the horizon moderates impact but decays expected alpha. Capacity is therefore determined jointly by liquidity, execution policy and the persistence of the underlying signal.

References

- [1] R. Rak et al. “Tests of scaling and universality of the distributions of trade size and share volume: Evidence from three distinct markets.” *Physical Review E* 76, 046109 (2007). doi:10.1103/PhysRevE.76.046109.
- [2] J. D. Farmer, A. Gerig, F. Lillo and H. Waelbroeck. “How efficiency shapes market impact.” *Quantitative Finance* 13(11), 1743–1758 (2013). arXiv:1102.5457.
- [3] R. Almgren and N. Chriss. “Optimal Execution of Portfolio Transactions.” *Journal of Risk* 3, 5–39 (2001). doi:10.21314/JOR.2001.041.
- [4] N. Gârleanu and L. H. Pedersen. “Dynamic Trading with Predictable Returns and Transaction Costs.” *Journal of Finance* 68(6), 2309–2340 (2013). SSRN 1444706.
- [5] R. Di Mascio, A. Lines and N. Y. Naik. “Alpha Decay and Institutional Trading.” Working paper, revised 2022. SSRN 2580551.